

Q. Compare and contrast Linked Lists and Arrays. Discuss the primary advantages and disadvantages of using a Linked List over an Array.

📌 Definition to Remember

Both **Arrays** and **Linked Lists** are linear data structures, but they differ fundamentally in memory allocation. Arrays use **static, contiguous** memory and allow random access. Linked Lists use **dynamic, non-contiguous** memory connected via pointers, allowing flexible sizing but restricting access to sequential traversal.

1. Array vs Linked List Comparison

Feature	Array	Linked List
Memory Allocation	Static (at compile time)	Dynamic (at runtime)
Memory Layout	Contiguous (side-by-side)	Non-contiguous (scattered)
Size	Fixed size	Dynamic size (grows/shrinks)
Access Speed	Fast O(1) (Random access via index)	Slow O(n) (Sequential access only)
Insert/Delete	Slow O(n) (Requires shifting elements)	Fast O(1) (Just pointer updates)
Memory Overhead	None	High (Must store a pointer per node)

2. Advantages of Linked List (over Array)

- **Dynamic Size:** Memory is allocated precisely as needed during runtime. No "array full" errors.
- **Efficient Insertions & Deletions:** Adding or removing an element anywhere (if the node is known) takes $O(1)$ time. No elements need to be shifted, unlike in an array.
- **No Memory Wastage:** Unused capacity is not reserved in advance.

3. Disadvantages of Linked List

- **Memory Overhead:** Every node requires extra space to store the memory address (pointer) of the next node.
- **No Random Access:** To access the n th element, you must traverse sequentially from the **head** to n . Binary Search cannot be efficiently applied.
- **Cache Unfriendly:** Because nodes are scattered in memory, CPU caching is less efficient compared to contiguous arrays.

★ Must-Write Points (for 10 marks)

1. Arrays use static, contiguous memory; Linked Lists use dynamic, non-contiguous memory.
2. Arrays have a fixed size; Linked lists can grow and shrink dynamically at runtime.
3. Arrays allow **O(1) random access** via index; Linked lists require **O(n) sequential traversal**.

4. Insertion/Deletion in arrays is slow ($O(n)$ shifting); in linked lists, it's fast (pointer updates).
5. Linked lists have **memory overhead** because each node must store a pointer.
6. Arrays are CPU cache-friendly; linked lists are not.
7. Choose Arrays for fast searching/accessing; choose Linked Lists for frequent insertions/deletions.

⚡ Quick Recall

Array (Static, Contiguous, Fixed size, $O(1)$ Access, Slow Insert/Delete) vs Linked List (Dynamic, Scattered, Variable size, $O(n)$ Access, Fast Insert/Delete, Pointer Overhead)